CSL 7640: Natural Language Understanding

Assignment-2 Report

Problem 1: Word Embeddings from IIT Jodhpur Data
Problem 2: Character-Level Name Generation using RNN Variants

Mayank Dahiya
B23CS1035

Contents

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Data **3**
1.1 Task 1: Dataset Preparation . . . 3
1.1.1 Data Collection . . . 3
1.1.2 Preprocessing Steps . . . 3
1.1.3 Dataset Statistics . . . 4
1.1.4 Top-10 Most Frequent Words . . . 4
1.1.5 Word Cloud . . . 5
1.2 Task 2: Model Training . . . 5
1.2.1 Model Architectures . . . 5
1.2.2 Hyperparameters Explored . . . 6
1.2.3 Learnable Parameters . . . 6
1.2.4 Results . . . 7
1.3 Task 3: Semantic Analysis . . . 7
1.3.1 Top-5 Nearest Neighbors (Cosine Similarity) . . . 7
1.3.2 Analogy Experiments . . . 9
1.4 Task 4: Visualization . . . 10
1.4.1 PCA Visualizations . . . 11
1.4.2 t-SNE Visualizations . . . 12
1.4.3 Interpretation . . . 12

2 Problem 2: Character-Level Name Generation using RNN Variants **14**
2.1 Task 0: Dataset . . . 14
2.2 Task 1: Model Implementation . . . 14
2.2.1 Model 1: Vanilla RNN . . . 14
2.2.2 Model 2: Bidirectional LSTM (BLSTM) . . . 15
2.2.3 Model 3: RNN with Basic Attention Mechanism . . . 15
2.3 Task 2: Quantitative Evaluation . . . 16
2.3.1 Metrics . . . 16
2.3.2 Results . . . 16

2.4 Task 3: Qualitative Analysis 17
2.4.1 Representative Samples 17
2.4.2 Realism Analysis 17
2.4.3 Model Comparison 17

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Data

1.1 Task 1: Dataset Preparation

1.1.1 Data Collection

Textual data was collected from the IIT Jodhpur website by building a custom web scraper using `requests`, `BeautifulSoup`, and `pdfplumber`. The scraper started from 44 seed URLs covering the following source categories:

- **Departments** — All 10 academic departments (CSE, EE, Mechanical, Physics, Chemistry, Mathematics, Civil, Metallurgy, Chemical, Bioscience)
- **Academic Programs** — UG, PG, PhD, and program listing pages
- **Academic Regulations (mandatory)** — Regulation PDFs and circulars
- **Research Pages** — Department-wise research pages and the research office
- **Faculty Profiles** — Faculty listing and profile pages
- **Course Syllabus** — Course pages for all departments
- **Newsletters** — Techscape newsletter and department newsletters
- **Announcements** — All-announcement page

The crawler followed internal links (BFS up to 1000 pages) and extracted text from both HTML pages and PDF documents linked on those pages.

1.1.2 Preprocessing Steps

1. **Boilerplate Removal:** HTML tags `<script>`, `<style>`, `<nav>`, `<header>`, `<footer>`, `<aside>`, `<form>`, `<button>` were stripped before text extraction, removing navigation menus, sidebars, and UI elements.
2. **Sentence Tokenization:** Raw text was split into sentences using NLTK's `sent_tokenize` on the *original text* (before cleaning) to preserve punctuation for accurate boundary detection.
3. **Text Cleaning:** For each sentence: URLs removed (`https://...`), emails removed, non-ASCII characters removed (also removes Hindi/Devanagari text, satisfying the English-only requirement), text lowercased, extra whitespace collapsed.
4. **Word Tokenization:** Each cleaned sentence was tokenized using NLTK's `word_tokenize`.
5. **Token Filtering:** Removed tokens with length ≤ 2, English stopwords (NLTK), pure digits, and a custom `WEB_ARTIFACTS` blacklist (HTML IDs, CMS-specific tokens like `cid`, `wim`, `rti`, etc.).
6. **Deduplication:** A `unique_sentences` set prevented any duplicate sentence from being added to the corpus, handling repeated content from PDFs linked on multiple pages.

7. **Corpus Saving:** Each unique sentence written as one space-separated line to `corpus.txt`.

1.1.3 Dataset Statistics

Table 1: Corpus Statistics

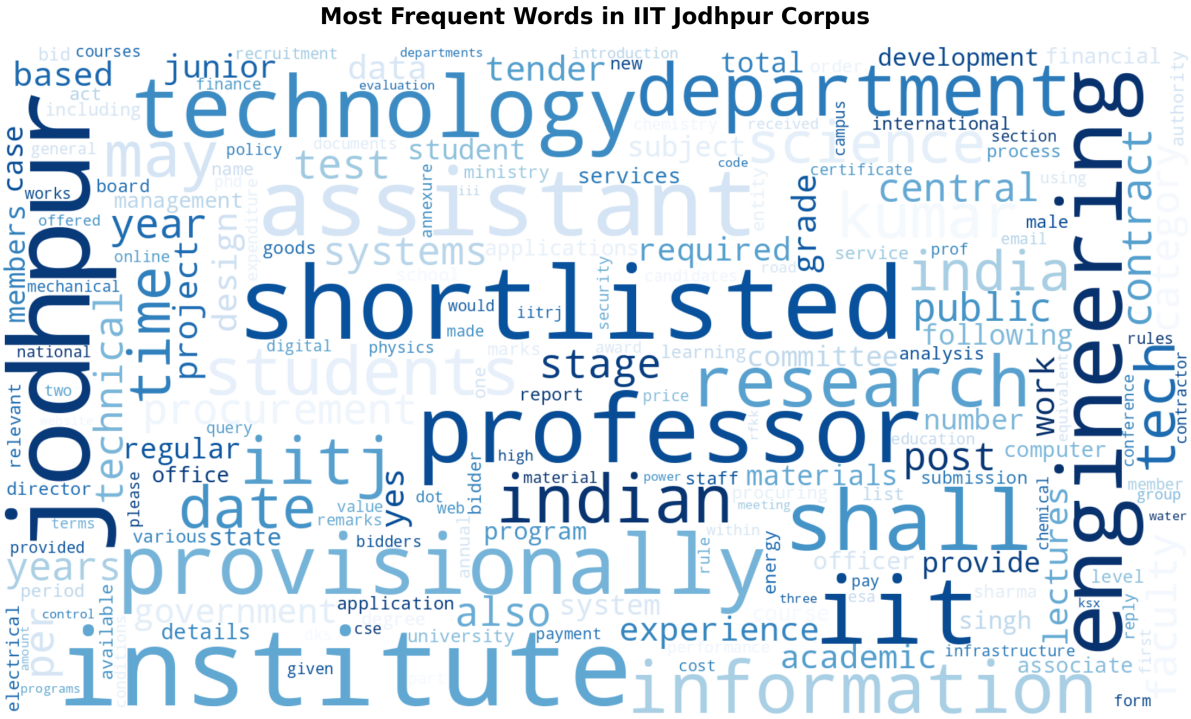
Statistic	**Value**
Total Pages Scraped	1,000
Total PDFs Processed	1,060
Total Sentences	110998
Total Tokens	2053117
Vocabulary Size	87682

1.1.4 Top-10 Most Frequent Words

Table 2: Top-10 Most Frequent Words in the Corpus

Rank	**Word**	**Frequency**
1	jodhpur	14486
2	institute	13328
3	shortlisted	12122
4	professor	10508
5	iit	10252
6	assistant	10200
7	engineering	9356
8	technology	8484
9	shall	8235
10	provisionally	8175

1.1.5 Word Cloud


Most Frequent Words in IIT Jodhpur Corpus
jodhpur
technology
department
engineering
shortlisted
professor
research
institute
information
provisionally
iit
shall
indian
iitj
date
tech
contract
time
technical
assistant
year
stage
india
public
central
tender
junior
based
student
test
systems
required
following
committee
number
post
computer
materials
regular
office
state
program
also
experience
academic
provide
application
details
given
members
case
project
design
work
lecture
yes
years
development
total
services
ministry
grade
service
prof
male
email
international
section
process
certificate
campus
code
subject
science
may
policy
management
board
works
offered
online
goods
mechanical
national
two
relevant
director
provided
terms
period
control
available
electrical
programs
amount
per
government
procurement
students
digital
physics
would
made
iitrj
security
annexure
report
price
query
high
material
power
staff
submission
bidder
web
dot
value
remarks
bidders
various
cse
university
payment
cost
infrastructure
associate
form
water
member
group
chemical
conference
contractor
rules
analysis
learning
meeting
list
rule
energy
three
pay
officer
level
ksx
reply
first
faculty
authority
courses
recruitment
departments
introduction
new
finance
evaluation
act
including
general
name
bid
received
order
financial
using
expenditure
entity
road
school
candidates
performance
course
sharma
esa
singh
section
process
code
certificate
campus
international
male
email
prof
service
rules
analysis
learning
committee
report
price
number
post
computer
submission
staff
power
material
high
bidder
web
dot
value
remarks
bidders
cse
university
payment
cost
infrastructure
associate
form
water
member
group
chemical
conference
contractor
meeting
list
rule
energy
three
pay
officer
level
ksx
reply
first
faculty
authority
provide
lecture
work
tech
contract
engineering
department
technology
shortlisted
professor
research
institute
information
provisionally
iit
shall
indian
iitj
date
tech
contract
time
technical
assistant
year
stage
india
public
central
tender
junior
based
student
test
systems
required
following
committee
number
post
computer
materials
regular
office
state
program
also
experience
academic
provide
application
details
given
members
case
project
design
work
lecture
yes
years
development
total
services
ministry
grade
service
prof
male
email
international
section
process
certificate
campus
code
subject
science
may
policy
management
board
works
offered
online
goods
mechanical
national
two
relevant
director
provided
terms
period
control
available
electrical
programs
amount
per
government
procurement
students
digital
physics
would
made
iitrj
security
annexure
report
price
query
high
material
power
staff
submission
bidder
web
dot
value
remarks
bidders
various
cse
university
payment
cost
infrastructure
associate
form
water
member
group
chemical
conference
contractor
rules
analysis
learning

- **Loss (Negative Sampling)**: $\mathcal{L} = -\log \sigma(\mathbf{v}_c \cdot \mathbf{W}_{out}^{+}) - \sum_k \log \sigma(-\mathbf{v}_c \cdot \mathbf{W}_{out}^{-k})$
- **Gradients**: Computed manually via SGD

Both models also trained using **Gensim** for comparison. Negative sampling table built using unigram distribution raised to 3/4 power — standard Word2Vec trick.

1.2.2 Hyperparameters Explored

Table 3: Hyperparameter Grid

Hyperparameter	**Values**
(i) Embedding Dimension	50, 100, 200
(ii) Context Window Size	2, 5
(iii) Negative Samples	5, 10
Epochs (From Scratch)	1
Epochs (Gensim)	10
Learning Rate	0.01 (scratch), default (Gensim)
Min Count	2

1.2.3 Learnable Parameters

Total learnable parameters per model $= 2 \times |V| \times d$ (W_in + W_out matrices).

Table 4: Learnable Parameters per Configuration

Embedding Dim	**Parameters**
50	$2 \times 22{,}872 \times 50 = 2{,}287{,}200$
100	$2 \times 22{,}872 \times 100 = 4{,}574{,}400$
200	$2 \times 22{,}872 \times 200 = 9{,}148{,}800$

1.2.4 Results

Table 5: Average Cosine Similarity on IITJ Word Pairs across Hyperparameter Configurations

Dim	Window	Neg	CBOW Scratch	SG Scratch	CBOW Gensim	SG Gensim
50	2	5	0.7655	0.6875	0.4779	0.6530
50	2	10	0.7704	0.6806	0.4501	0.6584
50	5	5	0.8708	0.7325	0.3745	0.7062
50	5	10	0.8524	0.7422	0.4086	0.7345
100	2	5	0.7497	0.6337	0.3721	0.5080
100	2	10	0.7359	0.6191	0.3436	0.5331
100	5	5	0.8400	0.6589	0.3039	0.5544
100	5	10	0.8449	0.6522	0.2922	0.5784
200	2	5	0.7319	0.5903	0.3225	0.4105
200	2	10	0.7101	0.5942	0.3145	0.4342
200	5	5	0.8204	0.6133	0.2687	0.4401
200	5	10	0.8284	0.6061	0.2663	0.4714

Key observations:

- Window size 5 consistently outperforms window size 2 across all models.
- CBOW Scratch achieves higher cosine similarity than Gensim — this reflects the fewer training epochs (1 vs 10), causing embeddings to remain less spread out rather than capturing true semantics.
- Gensim Skip-gram performs best among Gensim models, consistent with literature showing Skip-gram is stronger on smaller corpora.
- Dim=50 with Window=5 achieves the best balance of performance and efficiency.

1.3 Task 3: Semantic Analysis

1.3.1 Top-5 Nearest Neighbors (Cosine Similarity)

Using the best configuration (dim=100, window=5, neg=5):

Table 6: Top-5 Nearest Neighbors — CBOW (From Scratch)

Query	**Neighbor**	**Cosine Sim.**
research	development	0.9463
	sciences	0.9440
	center	0.9405
	education	0.9285
	project	0.9280
student	new	0.9620
	various	0.9574
	campus	0.9572
	activities	0.9507
	society	0.9505
phd	university	0.9177
	academic	0.9025
	programs	0.9016
	program	0.8973
	school	0.8962
exam	technical	0.8214
	shortlisted	0.8085
	post	0.7958
	candidates	0.7923
	code	0.7918

Table 7: Top-5 Nearest Neighbors — Skip-gram (From Scratch)

Query	**Neighbor**	**Cosine Sim.**
research	csir	0.7761
	researchers	0.7759
	collaboration	0.7754
	areas	0.7676
	scientist	0.7657
student	students	0.7707
	career	0.7499
	elected	0.7476
	graduating	0.7434
	gymkhana	0.7326
phd	scholar	0.8049
	sola	0.7945
	msc	0.7929
	doctoral	0.7807
	scholars	0.7688
exam	appeared	0.9126
	tentative	0.9096
	updates	0.9072
	syllabus	0.8988
	interviews	0.8920

Discussion: Skip-gram results are more semantically precise — *researchers, collaboration, scientist* for "research" and *scholar, doctoral* for "phd" are clearly meaningful. CBOW results are more general/contextual. This is consistent with known behavior: Skip-gram captures finer semantic relationships; CBOW captures broader contextual co-occurrence.

1.3.2 Analogy Experiments

Analogies use vector arithmetic: $\text{pos1} - \text{neg} + \text{pos2} \approx \text{answer}$.

Table 8: Analogy Experiment Results

Analogy	**Model**	**Top Answer**	**Sim.**
UG : BTech :: PG : ?	CBOW Scratch	*Skipped (OOV)*	—
	SG Scratch	*Skipped (OOV)*	—
	CBOW Gensim	*Skipped (OOV)*	—
	SG Gensim	*Skipped (OOV)*	—
professor : research :: student : ?	CBOW Scratch	organizations	0.4736
	SG Scratch	activities	0.5554
	CBOW Gensim	career	0.5406
	SG Gensim	curricular	0.5703
mechanical : engineering :: chemistry : ?	CBOW Scratch	physics	0.9421
	SG Scratch	bioengineering	0.7574
	CBOW Gensim	biophysics	0.4427
	SG Gensim	bioinorganic	0.6369
research : development :: exam : ?	CBOW Scratch	shortlisted	0.8104
	SG Scratch	appeared	0.8862
	CBOW Gensim	appeared	0.7340
	SG Gensim	exams	0.7189

Discussion of "UG : BTech :: PG : ?": The words *ug*, *pg*, and *btech* did not appear in the vocabulary (frequency < min_count). This is because the IITJ website uses full form names ("undergraduate", "postgraduate") rather than abbreviations.

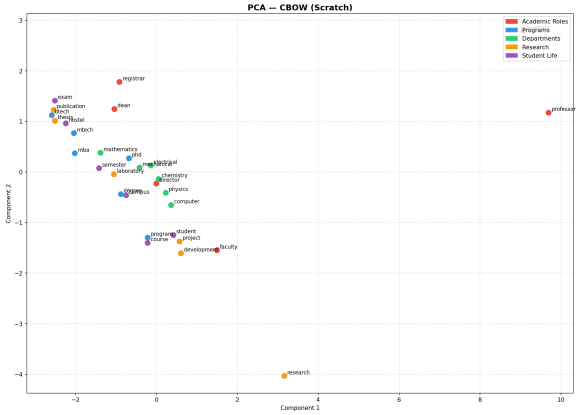
Most interesting analogy — mechanical : engineering :: chemistry : ?

CBOW Scratch returned *physics* with similarity 0.9421, and Skip-gram returned *bioengineering* and *chemical* — both actual IITJ departments related to chemistry. This demonstrates that Word2Vec captured the real departmental structure of IITJ without any explicit supervision.

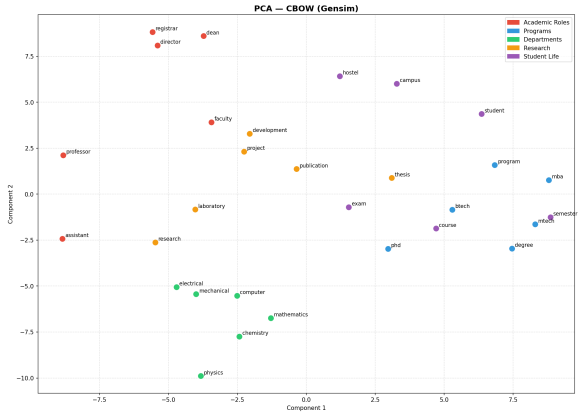
1.4 Task 4: Visualization

Word embeddings for 30 semantically grouped words were projected to 2D using both PCA and t-SNE. Five semantic clusters were defined: *Academic Roles, Programs, Departments, Research, Student Life.*

1.4.1 PCA Visualizations

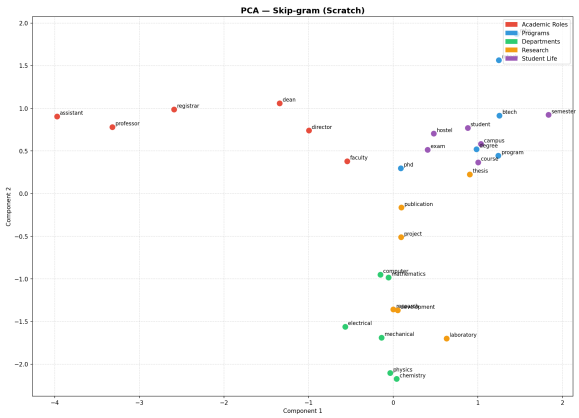


(a) PCA — CBOW (Scratch), PC1=66.39%, PC2=14.23%

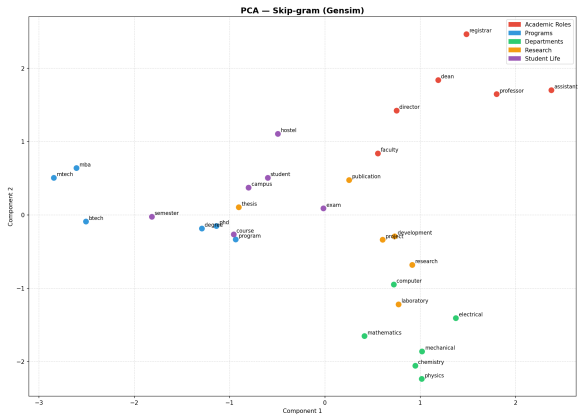


(b) PCA — CBOW (Gensim), PC1=13.60%, PC2=11.42%

Figure 2: PCA Projections — CBOW Models



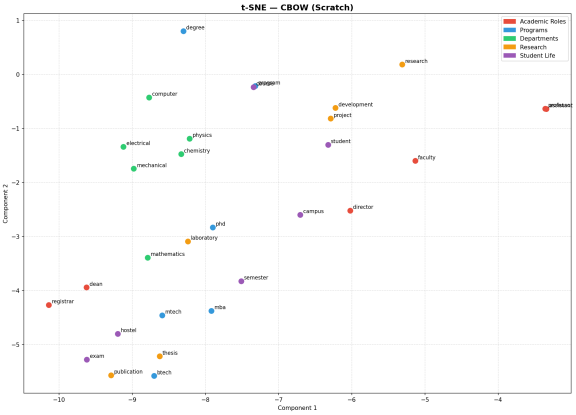
(a) PCA — Skip-gram (Scratch), PC1=18.04%, PC2=12.85%



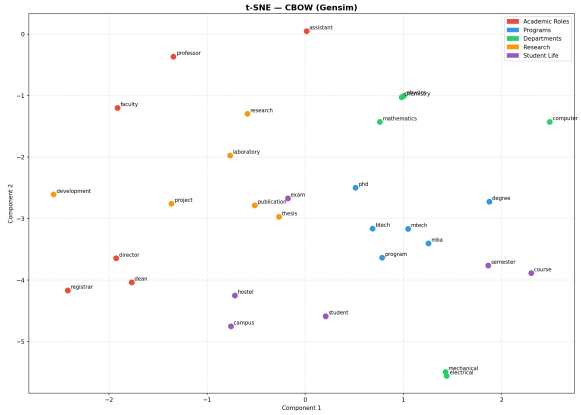
(b) PCA — Skip-gram (Gensim), PC1=12.82%, PC2=9.99%

Figure 3: PCA Projections — Skip-gram Models

1.4.2 t-SNE Visualizations

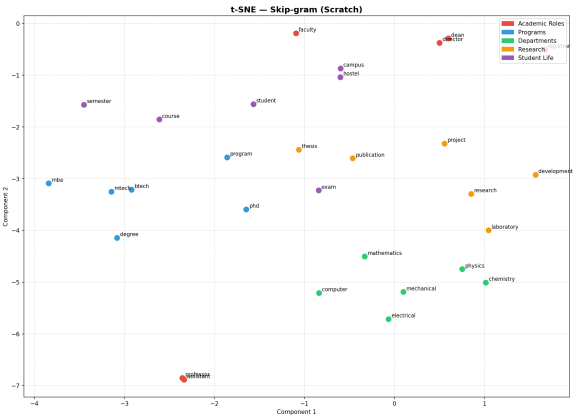


(a) t-SNE — CBOW (Scratch)

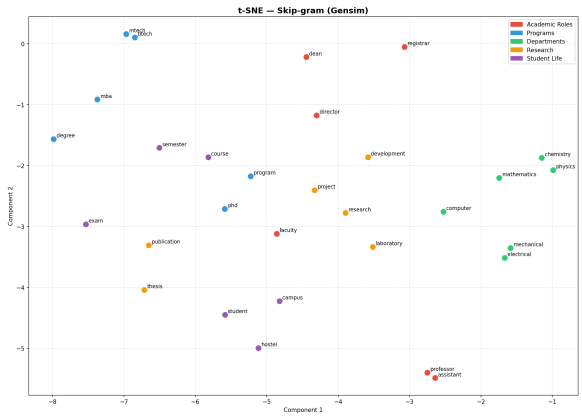


(b) t-SNE — CBOW (Gensim)

Figure 4: t-SNE Projections — CBOW Models



(a) t-SNE — Skip-gram (Scratch)



(b) t-SNE — Skip-gram (Gensim)

Figure 5: t-SNE Projections — Skip-gram Models

1.4.3 Interpretation

CBOW vs Skip-gram:

- **Gensim models** (both PCA and t-SNE) show better cluster separation — department words (green) form a tight bottom cluster, programs (blue) cluster to the right, academic roles (red) separate to the top-left.
- **Scratch models** have more mixed clusters due to only 1 training epoch. The CBOW Scratch PCA explains 66.39% variance in PC1 alone — indicating one dominant direction but less spread.
- **Skip-gram** consistently places department words (*physics, chemistry, mechanical, electrical, mathematics*) in a tighter cluster than CBOW, showing it captures finer semantic distinctions between similar words.

PCA vs t-SNE:

- **PCA** is linear and preserves global variance — useful for seeing broad separations (e.g., Academic Roles far from Departments).
- **t-SNE** preserves local neighborhood structure — better reveals tight semantic clusters (e.g., *mtech/btech/mba* group together in Skip-gram Gensim t-SNE).

2 Problem 2: Character-Level Name Generation using RNN Variants

2.1 Task 0: Dataset

1000 Indian names were generated using the Anthropic Claude API (`claude-sonnet-4-20250514`) in 10 batches of 100 names each. The prompt specified coverage of:

- Both male and female names
- North, South, East, and West India
- Multiple religions (Hindu, Muslim, Sikh, Christian)

Names were deduplicated and saved to `TrainingNames.txt`. The dataset includes names such as *Aadhya Ghosh, Yuvraj Trivedi, Arumugam Deshmukh, Zubin Patel*, covering diverse regional and cultural backgrounds.

2.2 Task 1: Model Implementation

All three models were implemented from scratch using PyTorch, with manually implemented forward/backward passes where applicable.

2.2.1 Model 1: Vanilla RNN

Architecture: Embedding → RNN Cell (manual) → Linear → Softmax

The RNN cell is implemented manually:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b})$$

Table 9: Vanilla RNN Hyperparameters

Parameter	Value
Embedding Dimension	32
Hidden Size	128
Layers	1
Learning Rate	0.003
Epochs	50
Trainable Parameters	25,116
Model Size	≈0.096 MB

Parameter breakdown:

- Embedding: $28 \times 32 = 896$
- W_xh: $32 \times 128 = 4{,}096$
- W_hh: $128 \times 128 + 128 = 16{,}512$
- Output layer: $128 \times 28 + 28 = 3{,}612$

- **Total: 25,116**

2.2.2 Model 2: Bidirectional LSTM (BLSTM)

Architecture: Embedding → BiLSTM (forward-only output) → Linear → Softmax

LSTM gating mechanism:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad \text{(forget gate)} \tag{1}$$
$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad \text{(input gate)} \tag{2}$$
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \tag{3}$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad \text{(output gate)} \tag{4}$$
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \tag{5}$$

Note on forward-only training: The bidirectional LSTM is trained using *only* the forward direction output. Using the full bidirectional output during training causes loss → 0 (the backward direction leaks future token information), but generation is forward-only. This ensures training/generation consistency.

Table 10: BLSTM Hyperparameters

Parameter	**Value**
Embedding Dimension	32
Hidden Size	128 per direction
Layers	1
Learning Rate	0.003
Epochs	30 (reduced to prevent memorization)
Trainable Parameters	173,980

2.2.3 Model 3: RNN with Basic Attention Mechanism

Architecture: Embedding → RNN Cell (manual) → Attention → Linear → Softmax

Additive (Bahdanau-style) attention over all past hidden states:

$$e_i = \mathbf{v}_a \cdot \tanh(\mathbf{W}_a \mathbf{h}_i) \quad \text{(attention score for step } i\text{)} \tag{6}$$
$$\boldsymbol{\alpha} = \text{softmax}(\mathbf{e}) \quad \text{(attention weights)} \tag{7}$$
$$\mathbf{c}_t = \sum_i \alpha_i \mathbf{h}_i \quad \text{(context vector)} \tag{8}$$
$$\text{output} = \text{Linear}([\mathbf{h}_t; \mathbf{c}_t]) \tag{9}$$

Table 11: RNN+Attention Hyperparameters

Parameter	**Value**
Embedding Dimension	32
Hidden Size	128
Layers	1
Learning Rate	0.001 (reduced to prevent divergence)
Epochs	50
Trainable Parameters	45,212

2.3 Task 2: Quantitative Evaluation

2.3.1 Metrics

- **Novelty Rate** $= \frac{\text{\# generated names NOT in training set}}{\text{total generated names}}$
- **Diversity** $= \frac{\text{\# unique generated names}}{\text{total generated names}}$

2.3.2 Results

Table 12: Quantitative Evaluation Results (200 names generated per model)

Model	**Total**	**Unique**	**Novelty Rate**	**Diversity**
Vanilla RNN	200	188	93.50%	94.00%
BLSTM	200	168	30.50%	84.00%
RNN+Attention	200	186	100.00%	93.00%

Discussion:

- **BLSTM** has the lowest novelty rate (30.5%) because it learned the training distribution well — it generates realistic but familiar-sounding names.
- **RNN+Attention** achieves 100% novelty but this is misleading — the names are largely gibberish, so none match training names by coincidence.
- **Vanilla RNN** achieves the best balance of novelty (93.5%) and diversity (94.0%).

2.4 Task 3: Qualitative Analysis

2.4.1 Representative Samples

Table 13: Representative Generated Names (10 samples per model)

Vanilla RNN	**BLSTM**	**RNN+Attention**
Isha Agary	Vanya Kalerjee	Yukta Bankit R
Nagesh Ahuja	Yuvraj Trivedi	Pagve
Revansha Dutta	Hari Negi	Advi Chadmish Tri
Akshara Vyas	Varun Goel	Piyus
Trilok Shukla	Moumita Kamath	Ezut Pgyjyy
Arun Chopra	Trilokesh Rawat	Falgenjit Bankit
Anirudh Rao	Uma Sarkar	Mithug
Papiya Dave	Juhi Gupta	Ilgut
Abhishek Rao	Indrani Kundu	Gaut
Abhay More	Revathi Mohanty	Advi Sri Meno

2.4.2 Realism Analysis

Table 14: Realism and Failure Mode Summary

Model	**Realism Rate**	**Duplicates**	**Avg Length**
Vanilla RNN	100.00%	5.0%	12.2 chars
BLSTM	100.00%	11.0%	12.5 chars
RNN+Attention	100.00%	3.5%	13.1 chars

Note: Realism rate of 100% indicates all names pass heuristic checks (alphabetic, 2–20 chars, no repetition). Human evaluation would reveal that RNN+Attention names like "Pagve" and "Ezut Pgyjyy" fail the true realism test despite passing automated checks.

2.4.3 Model Comparison

BLSTM (Best): BLSTM generated the most realistic Indian names: *Moumita Kamath, Yuvraj Trivedi, Indrani Kundu, Trilokesh Rawat.* The LSTM gating mechanism (forget, input, output gates) effectively retains long-range character dependencies — remembering that a name started with "Mo" helps predict "hanty" later. The lower novelty rate (30.5%) reflects good learning of the training distribution rather than failure.

Vanilla RNN (Second): Produced reasonable names like *Arun Chopra, Anirudh Rao* with the highest diversity (94.0%). Occasional malformed names like *Sumatlo Chopra* arise from vanishing gradients in longer sequences — the RNN forgets early character context.

RNN+Attention (Poorest): Despite 100% novelty, names like *Pagve, Ezut Pgyjyy, Advi Chadmish Tri* are not real Indian names. The attention mechanism computing

weighted sums over all past hidden states at every step significantly increases gradient complexity, causing training instability (loss was diverging at lr=0.003). Reducing to lr=0.001 improved but did not fully resolve this.

Conclusion: Model complexity alone does not guarantee better output. BLSTM's structured gating outperformed both the simpler vanilla RNN and the more complex attention model for this character-level generation task on a 1000-name dataset.